Architecture: raw data to dashboard, with both AI features

Credit Risk Portfolio Monitor · Python · Streamlit · scikit-learn · LightGBM · optional free-tier LLM planner · all figures computed locally with pandas

1 Raw data

data/raw: 5 CSVs as provided.

customers · loan_accounts · bureau_credit · repayment_history · collections_events

14,685 loan accounts.

Tool: pandas.read_csv

2 Ingestion

src/ingestion.py, src/features.py

Cleans 14 documented data-quality issues; no rows dropped.

LEFT joins on customer_id / account_id; row counts asserted.

Repayment (months 1-12) and bureau aggregated to account grain.

Tool: pandas, pyarrow

3 Analytics

src/analytics.py

Profiles by income, employment, region, age, loan type; cross-cuts; trends; 5 insights with a 'so what'.

Tool: pandas

4 Model

src/model.py

Logistic regression baseline + LightGBM. Customer-grouped split; GroupKFold tuning on train only; one test evaluation.

AUC, KS, Gini; TreeSHAP drivers per account; PSI monitoring.

Tool: scikit-learn, LightGBM

5 Dashboard

app/app.py: 7 views

Overview · Segments · Account · Ask the Portfolio · Anomaly review · Model · Data & methods.

Hosted on Streamlit Community Cloud; LLM key in its secrets store.

Tool: Streamlit, Plotly

AI feature 1: Ask the Portfolio (LLM question answering)

Question

Analyst types a question in the dashboard.

Guardrails

src/ask.py. Declines injection, raw-row, forecast, causal and off-topic questions before any planner runs.

Planner

LLM (if a key is set) or offline rules. Returns a JSON plan: metric, group-by, filters.

Validate + run

Plan checked against a closed list; pandas executes it locally.

Answer

Text rendered from the result; every number checked against it. Source and plan shown.

Where the LLM is called and what is sent

Call site: src/ask.py, LLMPlanner.plan() · one HTTPS request to an OpenAI-compatible chat endpoint (Groq, Gemini or OpenRouter free tiers), key read server-side from the secrets store.

SENT: the question text and the table catalog only (metric and column descriptions, category names, limits).

RETURNED: one JSON query plan.

NEVER SENT: row-level data, identifiers, computed results or the API key. No key, rate limit or error: the app falls back to the offline interpreter with a visible notice.

AI feature 2: Anomaly detection

Detect (src/anomaly.py, pandas)

Peer segment (loan type x income band): value above the group's 99th percentile. Own history: last 3 months vs first 9. Flag when 2+ signals, or a sharp own-history break.

Result: 238 of 14,685 accounts flagged (1.6%), each with reasons.

Review list (dashboard)

Analyst reads the reasons with their numbers, then records Confirm or Dismiss with a note.

Decision store (src/decisions.py, SQLite)

Timestamp, account id, decision, note, reviewer. Download as CSV. Never changes the model.

Data flow: raw CSVs -> ingestion -> account-grain dataset -> analytics + model + anomaly flags -> portfolio table -> dashboard. Reproducible with: pip install -r requirements.txt; python -m src.build; streamlit run app/app.py

Synthetic data. Protected attributes (gender, marital status) and collections events are kept out of the model.